

**Министерство науки и высшего образования Российской Федерации**
**Федеральное государственное автономное образовательное учреждение высшего образования**
**«Московский государственный технический университет имени Н.Э. Баумана**
**(национальный исследовательский университет)»**
**(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

# Лабораторная работа № 1
# по дисциплине «Анализ алгоритмов»

**Тема** Алгоритмы умножения матриц

**Студент** Атаев К.А.

**Группа** ИУ7-52Б

**Преподаватели** Волкова Л. Л., Строганов Д.В.

Москва, 2025

# СОДЕРЖАНИЕ

**ВВЕДЕНИЕ** . . . . . **4**

**1 Аналитическая часть** . . . . . **5**

1.1 Классический алгоритм умножения матриц . . . . . 5

1.2 Алгоритм Винограда . . . . . 5

1.2.1 Анализ вычислительной сложности . . . . . 5

1.2.2 Особенности реализации . . . . . 6

**2 Конструкторская часть** . . . . . **7**

2.1 Представление алгоритмов . . . . . 7

2.2 Модель вычислений . . . . . 11

2.3 Трудоёмкость алгоритмов . . . . . 11

2.3.1 Стандартный алгоритм умножения матриц . . . . . 11

2.3.2 Алгоритм Винограда . . . . . 12

2.3.3 Оптимизированный алгоритм Винограда . . . . . 13

**3 Технологическая часть** . . . . . **15**

3.1 Средства реализации . . . . . 15

3.2 Реализация алгоритмов . . . . . 15

3.3 Функциональные тесты . . . . . 15

3.4 Вывод . . . . . 16

**4 Исследовательская часть** . . . . . **17**

4.1 Демонстрация работы программы . . . . . 17

4.2 Время выполнения реализаций алгоритмов . . . . . 17

4.3 Вывод . . . . . 18

**ЗАКЛЮЧЕНИЕ** . . . . . **20**

**СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ** . . . . . **21**

# ВВЕДЕНИЕ

**Цель лабораторной работы:** исследование алгоритмов умножения матриц — классический алгоритм, алгоритм Винограда и оптимизированный алгоритм Винограда.

**Задачи лабораторной работы:**

1) описать алгоритмы умножения матриц: стандартный и Винограда;
2) представить блок-схемы трёх рассматриваемых алгоритмов: стандартного, классического Винограда и его оптимизированной версии;
3) ввести модель вычислений и выполнить оценку трудоёмкости алгоритмов на основе их схем или реализаций;
4) реализовать каждый алгоритм;
5) провести замеры процессорного времени выполнения алгоритмов;
6) выполнить сравнительный анализ алгоритмов на основе данных замеров;

# 1 Аналитическая часть

В данном разделе рассматриваются алгоритмы умножения матриц: классический алгоритм и алгоритм Винограда. Основное внимание уделяется их вычислительной сложности и особенностям реализации.

## 1.1 Классический алгоритм умножения матриц

Пусть имеются две матрицы: матрица $A$ размером $n \times m$ и матрица $B$ размером $m \times q$. Их произведение $C = AB$ представляет собой матрицу размером $n \times q$, элементы которой определяются следующим выражением (1.1):

$$c_{ij} = \sum_{k=1}^{m} a_{ik} \cdot b_{kj}, \quad i = 1, \ldots, n;\ j = 1, \ldots, q \tag{1.1}$$

Сложность данного алгоритма оценивается как $O(n \cdot m \cdot q)$ операций. В случае квадратных матриц порядка $n$ сложность составляет $O(n^3)$.

## 1.2 Алгоритм Винограда

Алгоритм Винограда основан на альтернативном способе вычисления скалярного произведения. Для матриц $A \in \mathbb{R}^{n \times m}$ и $B \in \mathbb{R}^{m \times q}$ элемент результирующей матрицы вычисляется по формуле (1.2):

$$c_{ij} = \sum_{k=1}^{m/2} (a_{i,2k-1} + b_{2k,j})(a_{i,2k} + b_{2k-1,j}) - R_i - S_j \tag{1.2}$$

где предварительно вычисляются вспомогательные величины (1.3 и 1.4):

$$R_i = \sum_{k=1}^{m/2} a_{i,2k-1} a_{i,2k}, \quad i = 1, \ldots, n \tag{1.3}$$

$$S_j = \sum_{k=1}^{m/2} b_{2k-1,j} b_{2k,j}, \quad j = 1, \ldots, q \tag{1.4}$$

### 1.2.1 Анализ вычислительной сложности

Количество операций в алгоритме Винограда:

— умножения: $n \cdot \frac{m}{2} + q \cdot \frac{m}{2} + n \cdot q \cdot \frac{m}{2} = \frac{m}{2}(n + q + n \cdot q)$

— сложения: $n \cdot (\frac{m}{2} - 1) + q \cdot (\frac{m}{2} - 1) + n \cdot q \cdot (2 \cdot \frac{m}{2} + 1) = n(\frac{m}{2} - 1) + q(\frac{m}{2} - 1) + n \cdot q \cdot (m + 1)$

Для квадратных матриц размера $n \times n$ ($m = n$, $q = n$):

— умножения: $\frac{n}{2}(n + n + n^2) = \frac{n^3}{2} + n^2$

— сложения: $n(\frac{n}{2} - 1) + n(\frac{n}{2} - 1) + n^2(n + 1) = \frac{3}{2}n^3 + \frac{3}{2}n^2 - 2n$

Таким образом, алгоритм Винограда сокращает количество умножений примерно в 2 раза по сравнению с классическим алгоритмом, но увеличивает количество сложений.

### 1.2.2 Особенности реализации

При нечетном значении $m$ требуется дополнительная обработка (1.5):

$$c_{ij} = \left[ \sum_{k=1}^{\lfloor m/2 \rfloor} (a_{i,2k-1} + b_{2k,j})(a_{i,2k} + b_{2k-1,j}) - R_i - S_j \right] + a_{i,m} \cdot b_{m,j} \tag{1.5}$$

где $R_i$ и $S_j$ вычисляются для $k = 1, \ldots, \lfloor m/2 \rfloor$.

## Вывод

Алгоритм Винограда демонстрирует выигрыш в количестве операций умножения по сравнению с классическим алгоритмом, однако требует дополнительных операций сложения и предварительных вычислений. Эффективность алгоритма Винограда проявляется на больших матрицах, где сокращение количества умножений компенсирует увеличение сложности за счет дополнительных операций.

# 2 Конструкторская часть

В данном разделе представлены схемы алгоритмов умножения матриц и выполнена оценка их трудоёмкости.

## 2.1 Представление алгоритмов

Алгоритм принимает на вход две матрицы: $mtr1$ размера $N \times M$, $mtr2$ размера $M \times T$, где $M, N, T$ — неотрицательные целые числа; на выходе — результат умножения двух матриц, записанный в $res$ размера $M \times K$.

Оптимизированная версия алгоритма Винограда предусматривает следующие изменения:

— использование побитового сдвига целого числа вместо умножения на 2;

— объеденение IV и III частей алгоритма, избавившись от совпадающей пары циклов;

На рис. 2.1–2.3 приведены схемы стандартного алгоритма умножения матриц, алгоритма Винограда и оптимизированного алгоритма Винограда.

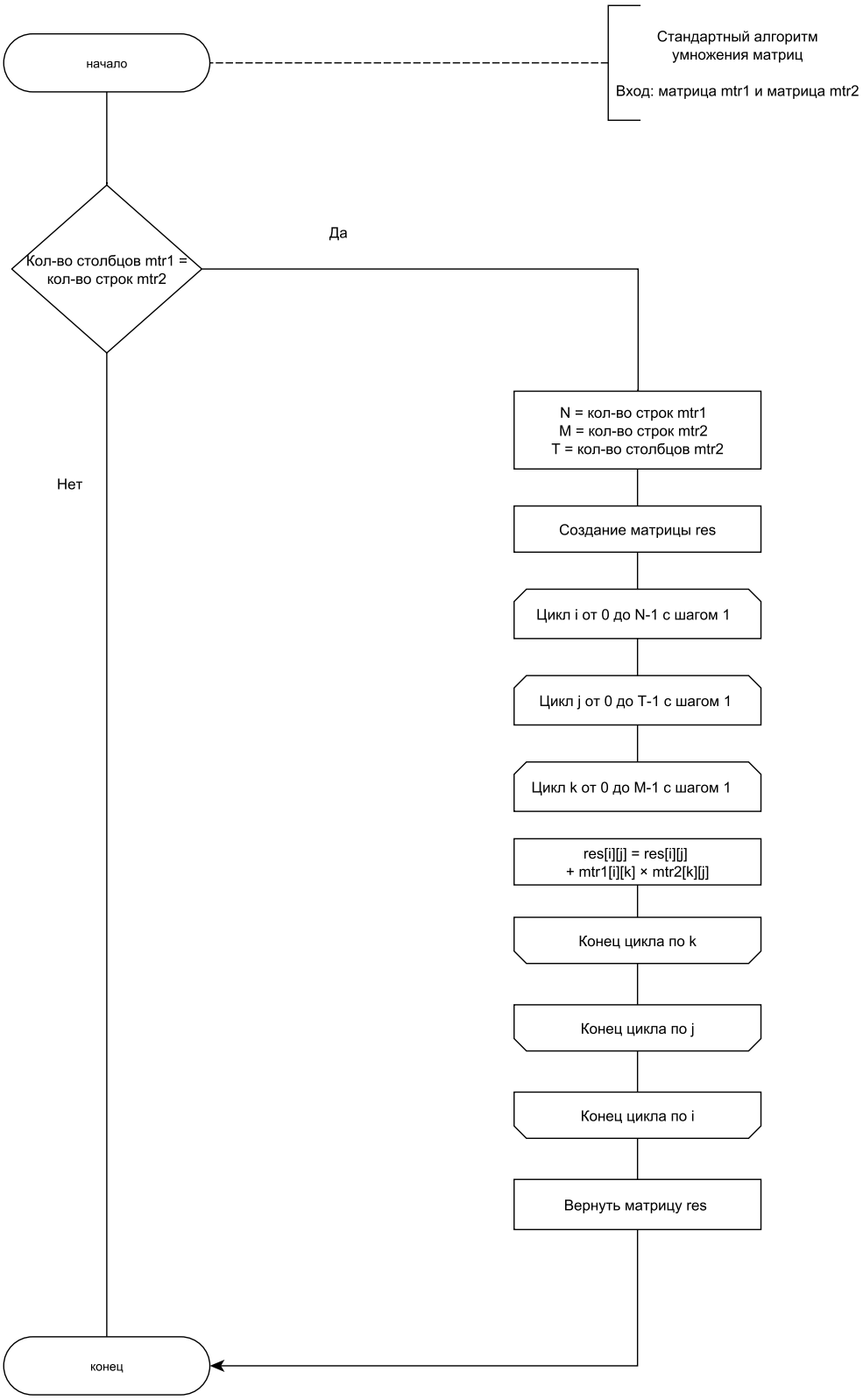



Рисунок 2.1 — Схема классического алгоритма умножения матриц

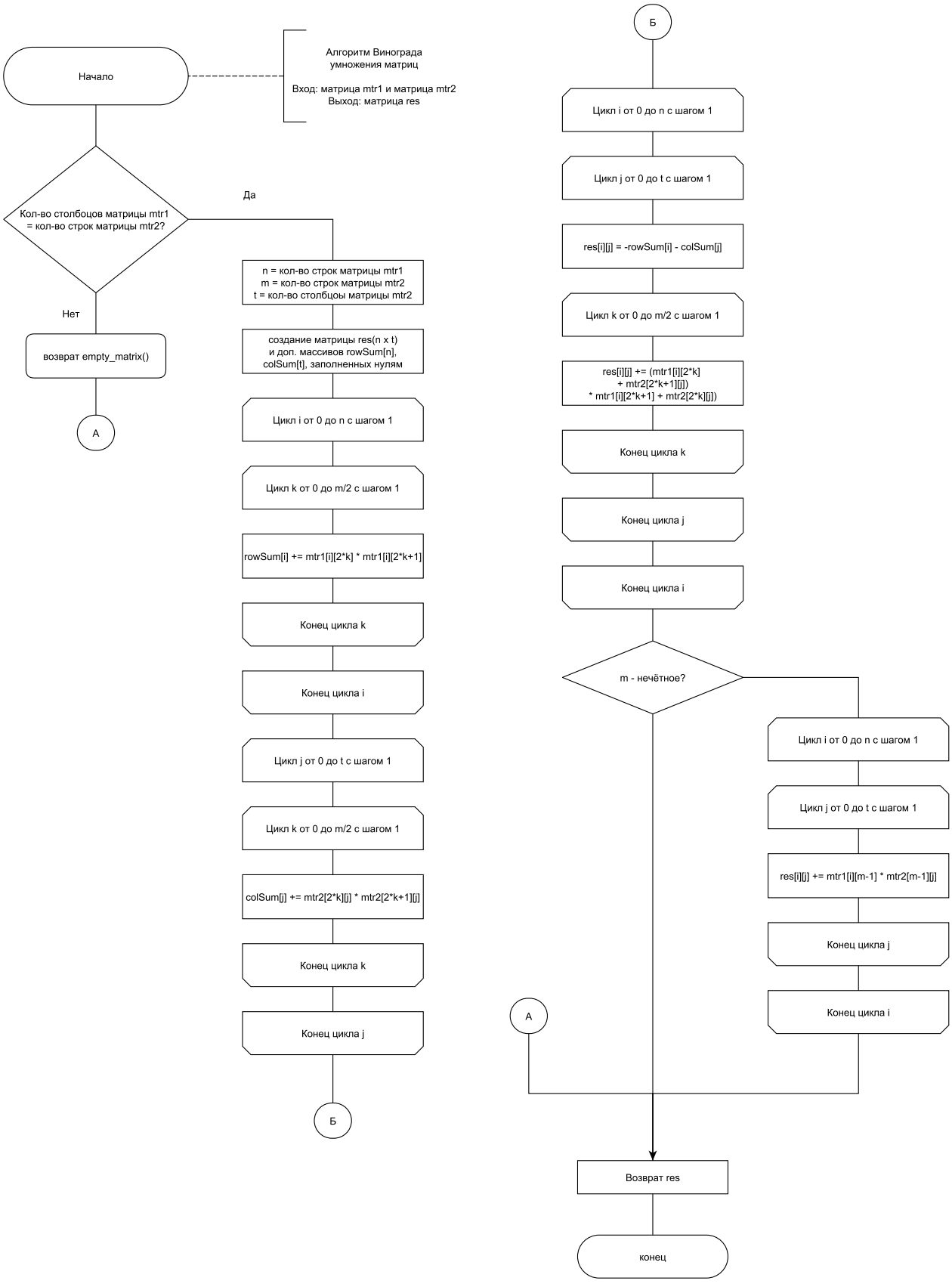



Рисунок 2.2 — Схема алгоритма Винограда умножения матриц

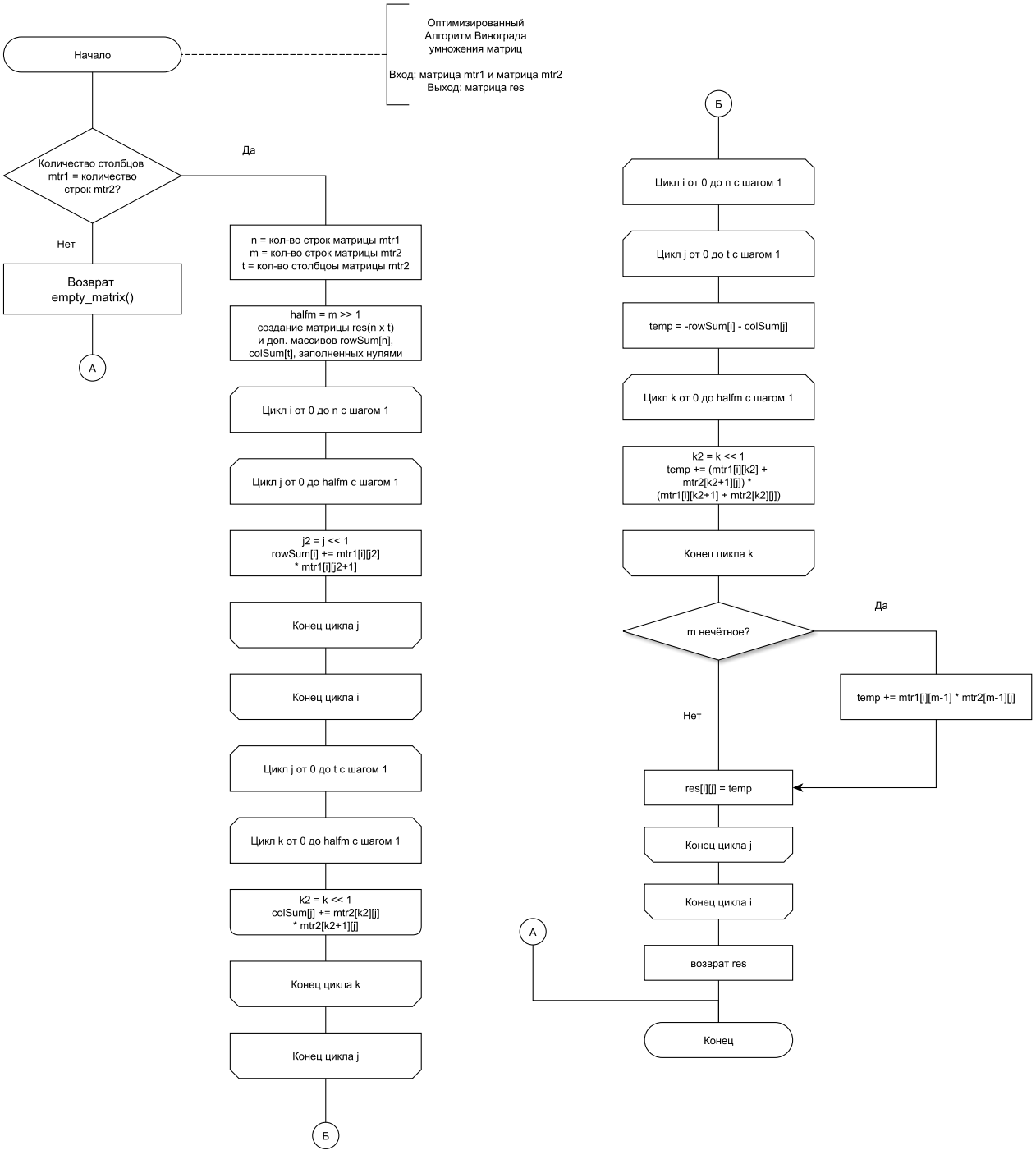



Рисунок 2.3 — Схема оптимизированного алгоритма Винограда умножения матриц

## 2.2 Модель вычислений

Для последующего вычисления трудоемкости необходимо ввести модель вычислений:

1) операции из списка (2.1) имеют трудоёмкость 1;

$$+, -, *, /, \%, ==, !=, <, >, <=, >=, [], ++, --\tag{2.1}$$

2) трудоёмкость условного оператора **if** рассчитывается по формуле (2.2);

$$f_{if} = f_{\text{условия}} + \begin{cases} \min(f_A, f_B), & \text{если условие выполняется,} \\ \max(f_A, f_B), & \text{иначе.} \end{cases}\tag{2.2}$$

3) трудоёмкость цикла рассчитывается как (2.3);

$$f_{for} = f_{\text{инициализации}} + f_{\text{сравнения}} + N(f_{\text{тела}} + f_{\text{инкремента}} + f_{\text{сравнения}})\tag{2.3}$$

4) трудоёмкость вызова функции/возврата результата равна 0;

## 2.3 Трудоёмкость алгоритмов

В следующих частях будут расчитаны трудоемкости алгоритмов умножения матриц. Также не будет учитываться инициализация матрицы результата, потому что данное действие есть во всех алгоритмах и при этом не является самым трудоёмким.

### 2.3.1 Стандартный алгоритм умножения матриц

Для расчёта трудоёмкости воспользуемся следующими обозначениями:

— n — кол-во строк первой матрицы;

— m — кол-во столбцов первой матрицы и кол-во строк второй матрицы;

— t — кол-во столбцов второй матрицы;

Трудоёмкость стандартного алгоритма умножения матриц состоит из:

— внешнего цикла по $i \in [0..n)$, трудоёмкость которого: $f = 2 + n \cdot (2 + f_{\text{body}})$;

— цикла по $j \in [0..t)$, трудоёмкость которого: $f = 2 + t \cdot (2 + f_{\text{body}})$;

— цикла по $k \in [0..m)$, трудоёмкость которого: $f = 2 + 10 \cdot m$;

Учитывая, что трудоёмкость стандартного алгоритма равна трудоёмкости внешнего цикла:

$$f_{\text{classic}} = 2 + n \cdot \big(4 + m \cdot (4 + 10t)\big) = 2 + 4n + 4nm + 10nmt\tag{2.4}$$

Для квадратных матриц ($n = m = t$) формула (2.4) принимает вид:

$$f_{\text{classic}} = 2 + 4n + 4n^2 + 10n^3\tag{2.5}$$

### 2.3.2 Алгоритм Винограда

Чтобы вычислить трудоёмкость алгоритма Винограда, нужно учесть следующее:

— трудоёмкости создания и инициализации вспомогательных массивов `rowSum` и `colSum`, равной (2.6);

$$f_{\text{init}} = n + t \tag{2.6}$$

— трудоёмкости заполнения массива `rowSum`, равной (2.7);

$$f_{\text{rowSum}} = 2 + n\left(2 + \frac{m}{2} \cdot 11\right) \tag{2.7}$$

— трудоёмкости заполнения массива `colSum`, равной (2.8);

$$f_{\text{colSum}} = 2 + t\left(2 + \frac{m}{2} \cdot 11\right) \tag{2.8}$$

— трудоёмкости основного цикла вычисления результата для чётной части внутренней размерности, равной (2.9);

$$f_{\text{cycle}} = 2 + n\left(4 + t\left(11 + \frac{m}{2} \cdot 23\right)\right) \tag{2.9}$$

— трудоёмкости дополнительного цикла при нечётной внутренней размерности, равной (2.10);

$$f_{\text{last}} = \begin{cases} 2, & \text{если } m \text{ чётно,} \\ 4 + n\left(4 + 14t\right), & \text{если } m \text{ нечётно.} \end{cases} \tag{2.10}$$

Подставив формулы (2.6)–(2.10), получаем итоговую трудоёмкость алгоритма Винограда в худшем случае ($m$ нечётно):

$$f_{\text{Винограда}} = 13 + 8n + 4t + \frac{11}{2}nm + \frac{11}{2}tm + 11nt + \frac{23}{2}nmt + 4 + 4n + 14nt \tag{2.11}$$

Упростим выражение:

$$f_{\text{Винограда}} = 17 + 12n + 4t + \frac{11}{2}nm + \frac{11}{2}tm + 25nt + \frac{23}{2}nmt \tag{2.12}$$

Для квадратных матриц ($n = m = t$) формула (2.12) принимает вид:

$$f_{\text{Винограда}} = 17 + 12n + 4n + \frac{11}{2}n^2 + \frac{11}{2}n^2 + 25n^2 + \frac{23}{2}n^3 \tag{2.13}$$

$$f_{\text{Винограда}} = 17 + 16n + 36n^2 + \frac{23}{2}n^3 \tag{2.14}$$

### 2.3.3 Оптимизированный алгоритм Винограда

Оптимизация заключается в:

— использовании побитового сдвига вместо умножения на 2;

— объединении цикла для нечётного размера матрицы с основным циклом — дополнительные вычисления при нечётном значении $m$ теперь выполняются внутри него;

— замене операций деления и взятия остатка на побитовые операции (`m » 1` вместо `m / 2`, `m & 1` вместо `m % 2`);

Тогда трудоёмкость оптимизированного алгоритма Винограда состоит из:

— трудоёмкости предварительного вычисления значения `halfm = m » 1`, равной (2.15);

$$f_{\text{half}} = 1 \tag{2.15}$$

— трудоёмкости создания и инициализации массивов `rowSum`, `colSum` и результата `res`, равной (2.16);

$$f_{\text{init}} = 5 + 2n + 2t \tag{2.16}$$

— трудоёмкости заполнения массива `rowSum`, равной (2.17);

$$f_{\text{rowSum}} = 2 + n\left(2 + \frac{m}{2} \cdot 10\right) : \tag{2.17}$$

— трудоёмкости заполнения массива `colSum`, равной (2.18);

$$f_{\text{colSum}} = 2 + t\left(2 + \frac{m}{2} \cdot 10\right) \tag{2.18}$$

— трудоёмкости основного цикла вычисления результата, включающего обработку нечётного случая, равной (2.19);

$$f_{\text{cycle}} = 2 + n\left(4 + t\left(10 + \frac{m}{2} \cdot 22 + f_{\text{odd}}\right)\right), \tag{2.19}$$

где $f_{\text{odd}}$ — трудоёмкость дополнительных операций при нечётном $m$, равная:

$$f_{\text{odd}} = \begin{cases} 0, & \text{если } m \text{ чётно}, \\ 4, & \text{если } m \text{ нечётно}. \end{cases} \tag{2.20}$$

Подставив формулы (2.15)–(2.20), получаем итоговую трудоёмкость оптимизированного алгоритма Винограда в худшем случае ($m$ нечётно):

$$f_{\text{опт. Винограда}} = 12 + 8n + 4t + 5nm + 5tm + 14nt + \frac{23}{2}nmt \tag{2.21}$$

Для квадратных матриц ($n = m = t$) формула (2.21) принимает вид:

$$f_{\text{опт. Винограда}} = 9 + 11n + 19n^2 + \frac{9}{2}n^3 \tag{2.22}$$

## Вывод

В данном разделе были представлены схемы алгоритмов умножения матриц, а также были проведены расчёты трудоёмкости каждого из трём алгоритмов.

# 3 Технологическая часть

В данном разделе описаны инструменты, использованные для реализации, а также представлены листинги трёх алгоритмов умножения матриц: стандартного, классического алгоритма Винограда и его оптимизированной версии.

## 3.1 Средства реализации

Для реализации был выбран язык C++ в качестве среды разработки использовалась Visual Studio Code с компилятором g++. Для модульного тестирования применялась библиотека Check, а для визуализации результатов — утилита gnuplot.

## 3.2 Реализация алгоритмов

В листингах 4.1 —4.3 представлены реализации алгоритмов умножения матриц.

## 3.3 Функциональные тесты

Для проверки корректности реализации алгоритмов умножения матриц были разработаны функциональные тесты, охватывающие различные классы эквивалентности. Результаты тестирования представлены в таблице 3.1.

Таблица 3.1 — Функциональные тесты алгоритмов умножения матриц

| **Входная матрица A** | **Входная матрица B** | **Ожидаемый результат** |
|---|---|---|
| $\begin{pmatrix}1 & 2\\3 & 4\end{pmatrix}$ | $\begin{pmatrix}5 & 6\\7 & 8\end{pmatrix}$ | $\begin{pmatrix}19 & 22\\43 & 50\end{pmatrix}$ |
| $\begin{pmatrix}1 & 2\\3 & 4\\5 & 6\end{pmatrix}$ | $\begin{pmatrix}7 & 8 & 9\\10 & 11 & 12\end{pmatrix}$ | $\begin{pmatrix}27 & 30 & 33\\61 & 68 & 75\\95 & 106 & 117\end{pmatrix}$ |
| $\begin{pmatrix}1 & 2\\3 & 4\end{pmatrix}$ | $\begin{pmatrix}1 & 0\\0 & 1\end{pmatrix}$ | $\begin{pmatrix}1 & 2\\3 & 4\end{pmatrix}$ |
| $\begin{pmatrix}1 & 2\\3 & 4\end{pmatrix}$ | $\begin{pmatrix}0 & 0\\0 & 0\end{pmatrix}$ | $\begin{pmatrix}0 & 0\\0 & 0\end{pmatrix}$ |
| $\begin{pmatrix}5\end{pmatrix}$ | $\begin{pmatrix}7\end{pmatrix}$ | $\begin{pmatrix}35\end{pmatrix}$ |
| $\begin{pmatrix}1 & 2 & 3\end{pmatrix}$ | $\begin{pmatrix}4\\5\\6\end{pmatrix}$ | $\begin{pmatrix}32\end{pmatrix}$ |
| $\begin{pmatrix}1\\2\\3\end{pmatrix}$ | $\begin{pmatrix}4 & 5 & 6\end{pmatrix}$ | $\begin{pmatrix}4 & 5 & 6\\8 & 10 & 12\\12 & 15 & 18\end{pmatrix}$ |
| $\begin{pmatrix}1 & 2 & 3\end{pmatrix}$ | $\begin{pmatrix}1 & 2\\3 & 4\end{pmatrix}$ | Сообщение об ошибке (несовместимые размеры) |

Разработанные тесты охватывают следующие классы эквивалентности:

1) умножение квадратных матриц одинаковой размерности;

2) умножение прямоугольных матриц различных размеров;
3) умножение на единичную матрицу (проверка нейтрального элемента);
4) умножение на нулевую матрицу (проверка поглощающего элемента);
5) умножение матриц-векторов (граничные случаи размерности);
6) обработка несовместимых размеров матриц;

Все тестовые случаи успешно пройдены, реализация всех трёх алгоритмов корректна.

## 3.4 Вывод

В технологической части представлены листинги реализаций алгоритмов умножения матриц, описаны выбранные средства разработки и приведены функциональные тесты. Тестирование подтвердило корректность работы всех реализованных алгоритмов для различных классов входных данных.

# 4 Исследовательская часть

В данном разделе будут приведены примеры работы программа, а также проведен сравнительный анализ алгоритмов при различных ситуациях на основе полученных данных.

Технические характеристики устройства, на котором выполнялось тестирование:

— операционная система: Windows 11 домашняя;

— память: 8 Гбайт;

— процессор: Intel(R) Core(TM) i5-10300H CPU @ 2.50ГГц, 2496 МГц, ядер: 4.

При тестировании ноутбук был включен в сеть электропитания. Во время тестирования ноутбук не был нагружен сторонними приложениями, которые как-то могли повлиять на результаты тестирования.

## 4.1 Демонстрация работы программы

На рисунке 4.1 приведен пример работы программы.

```
Ввод матрицы A:
Введите размер матрицы: 4 3
Введите элементы матрицы (4 строк по 3 элементов):
1 0 1
3 1 1
0 2 4
2 1 3
Ввод матрицы B:
Введите размер матрицы: 3 5
Введите элементы матрицы (3 строк по 5 элементов):
2 1 0 3 1
0 2 1 1 4
1 0 2 2 0

Результаты умножения матриц:
Классический алгоритм:
3 1 2 5 1
7 5 3 12 7
4 4 10 10 8
7 4 7 13 6

Алгоритм Винограда:
3 1 2 5 1
7 5 3 12 7
4 4 10 10 8
7 4 7 13 6

Оптимизированный алгоритм Винограда:
3 1 2 5 1
7 5 3 12 7
4 4 10 10 8
7 4 7 13 6
```

Рисунок 4.1 — Пример работы программы

## 4.2 Время выполнения реализаций алгоритмов

В таблице 4.1 приведены результаты замеров процессорного времени выполнения реализаций алгоритмов умножения матриц.

Таблица 4.1 — Результаты замеров времени для квадратных матриц

| Размер | Стандартный, мс | Виноград, мс | Опт. Виноград, мс |
|---|---|---|---|
| 20 | 0.087 | 0.080 | 0.068 |
| 40 | 0.623 | 0.536 | 0.447 |
| 60 | 2.074 | 1.749 | 1.447 |
| 80 | 4.902 | 4.117 | 3.392 |
| 100 | 9.520 | 7.950 | 6.600 |
| 120 | 16.503 | 13.762 | 11.380 |
| 140 | 26.156 | 21.719 | 17.910 |
| 160 | 39.461 | 32.870 | 27.088 |
| 180 | 56.482 | 47.040 | 38.601 |
| 200 | 76.937 | 64.044 | 52.667 |
| 220 | 102.094 | 85.188 | 70.015 |
| 240 | 133.548 | 112.311 | 92.565 |
| 260 | 171.101 | 144.612 | 119.971 |
| 280 | 214.758 | 181.387 | 150.638 |

По таблице 4.1 были построены графики представленные на рисунке 4.2.

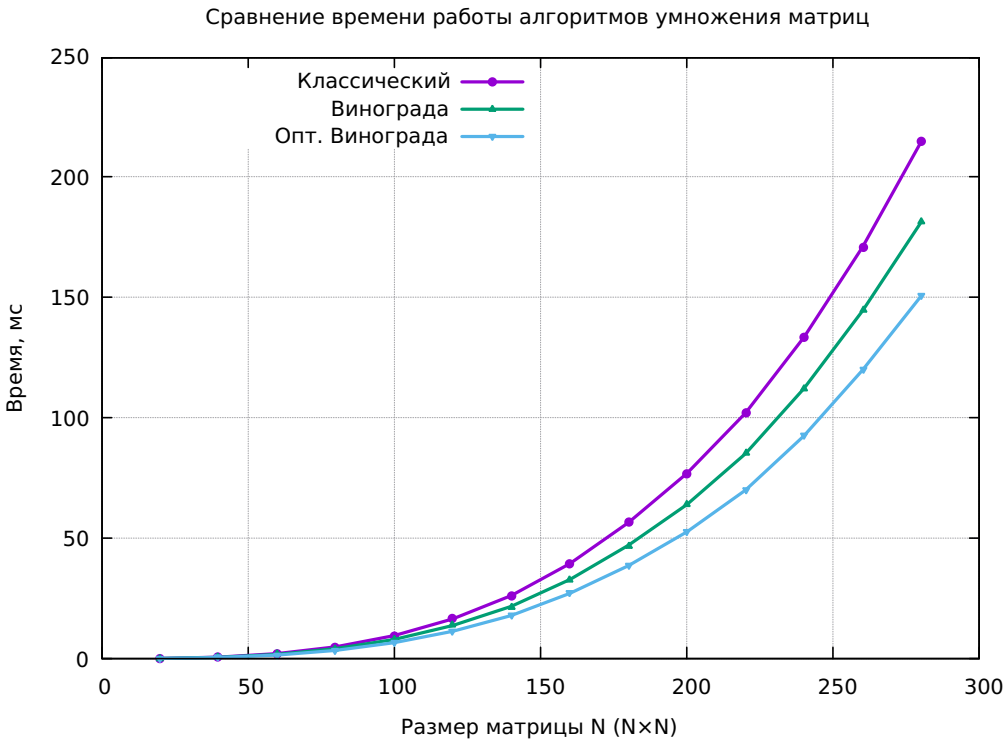



Рисунок 4.2 — Графики времени выполнения алгоритмов умножения матриц

## 4.3 Вывод

Были выполнены замеры процессорного времени для реализаций алгоритмов умножения матриц. Анализ полученных данных показал, что алгоритм Винограда превосходит классический по скорости в 1.09 раза на матрицах размером 10×10 и в 1.20 раза — на матрицах 100×100.

Оптимизированный алгоритм Винограда демонстрирует ещё большее преимущество, работая быстрее классического на 1.28 —1.46 раза в зависимости от размера матриц.

# ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были реализованны и исследованы алгоритмы умножения матриц. Все цели лабораторной работы были достигнуты, а задачи выполнены.

1) описаны алгоритмы умножения матриц: стандартный и Винограда.

2) представлены блок-схемы трёх рассматриваемых алгоритмов: стандартного, классического Винограда и его оптимизированной версии.

3) введена модель вычислений и выполнена оценка трудоёмкости алгоритмов на основе их схем или реализаций.

4) реализован каждый алгоритм.

5) проведены замеры процессорного времени выполнения алгоритмов.

6) выполнены сравнительный анализ алгоритмов на основе данных замеров.

Такое преимущество достигается благодаря двум ключевым улучшениям:

1) объединение циклов — исключены избыточные проходы по данным, что снижает накладные расходы на управление итерациями.

2) замена операций умножения на битовые сдвиги — операция сдвига (`<< 1`) выполняется процессором значительно быстрее, чем умножение на 2.

Наилучшие результаты продемонстрировал оптимизированный алгоритм Винограда: он оказался быстрее классического варианта примерно в 1,5 раза и превосходит базовую версию алгоритма Винограда почти в 1,2 раза.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ


1. Working Draft, Standard for Programming Language C++ [Электронный ресурс] // Open-std : [сайт]. — Режим доступа: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf (дата обращения: 24.10.2025)

2. GNU Compiler Collection (GCC) documentation [Электронный ресурс] // GCC : [сайт]. — Режим доступа: https://gcc.gnu.org/onlinedocs/ (дата обращения: 26.10.2025)

3. Intel® Core™ i5-10300H Processor [Электронный ресурс] // Intel : [сайт]. — Режим доступа: https://ark.intel.com/content/www/us/en/ark/products/201837/intel-core-i5-10300h-processor-8m-cache-up-to-4-50-ghz.html (дата обращения: 29.10.2025)

4. Головашкин, Д. Л. Векторные алгоритмы вычислительной линейной алгебры: учеб. пособие. — Самара: Изд-во Самарского университета, 2019. — С. 28–35

# ПРИЛОЖЕНИЕ А

```
matrix_v standart_multy(const matrix_v &mtr1, const matrix_v &mtr2)
{
  if (mtr1.empty() || mtr2.empty() || mtr1[0].empty() || mtr2[0].
     empty())
  return empty_matrix();

  if (mtr1[0].size() != mtr2.size())
  return empty_matrix();

  size_t n = mtr1.size();
  size_t m = mtr2.size();
  size_t t = mtr2[0].size();

  matrix_v res;
  array_v row;
  row.resize(t, 0);
  res.resize(n, row);

  for (size_t i = 0; i < n; i++)
  {
    for (size_t j = 0; j < t; j++)
    {
      for (size_t k = 0; k < m; k++)
      {
        res[i][j] += mtr1[i][k] * mtr2[k][j];
      }
    }
  }

  return res;
}
```

Листинг 4.1 — Реализация стандартного алгоритма умножения матриц

```
matrix_v vinograd_alg(const matrix_v &mtr1, const matrix_v &mtr2)
{
  if (mtr1.empty() || mtr2.empty() || mtr1[0].empty() || mtr2[0].
     empty())
  return empty_matrix();

  if (mtr1[0].size() != mtr2.size())
  return empty_matrix();

  size_t n = mtr1.size();
  size_t m = mtr2.size();
  size_t t = mtr2[0].size();

  matrix_v res;
  array_v rowSum, colSum;
  rowSum.resize(n, 0);
  colSum.resize(t, 0);

  array_v row;
  row.resize(t, 0);
  res.resize(n, row);

  for (size_t i = 0; i < n; i++)
  {
    for (size_t j = 0; j < m / 2; j++)
    {
      rowSum[i] += mtr1[i][2 * j] * mtr1[i][2 * j + 1];
    }
  }

  for (size_t i = 0; i < t; i++)
  {
    for (size_t j = 0; j < m / 2; j++)
    {
      colSum[i] += mtr2[2 * j][i] * mtr2[2 * j + 1][i];
    }
  }

  for (size_t i = 0; i < n; i++)
  {
```

```
      for (size_t j = 0; j < t; j++)
      {
        res[i][j] = -rowSum[i] - colSum[j];
        for (size_t k = 0; k < m / 2; k++)
        {
          res[i][j] += (mtr1[i][2 * k] + mtr2[2 * k + 1][j]) *
          (mtr1[i][2 * k + 1] + mtr2[2 * k][j]);
        }
      }
    }

    if (m % 2 == 1)
    {
      for(size_t i = 0; i < n; i++)
      {
        for(size_t j = 0; j < t; j++)
        {
          res[i][j] += mtr1[i][m - 1] * mtr2[m - 1][j];
        }
      }
    }

    return res;
  }
```

Листинг 4.2 — Реализация алгоритма Винограда

```
matrix_v opt_vinograd_alg(const matrix_v &mtr1, const matrix_v &mtr2)
{
  if (mtr1.empty() || mtr2.empty() || mtr1[0].empty() || mtr2[0].
     empty())
  return empty_matrix();

  if (mtr1[0].size() != mtr2.size())
  return empty_matrix();

  size_t n = mtr1.size();
  size_t m = mtr2.size();
  size_t t = mtr2[0].size();
  size_t halfm = m >> 1;

  matrix_v res;
  array_v rowSum, colSum;
  rowSum.resize(n, 0);
  colSum.resize(t, 0);

  array_v row;
  row.resize(t, 0);
  res.resize(n, row);

  for (size_t i = 0; i < n; i++)
  {
    for (size_t j = 0; j < halfm; j++)
    {
      size_t j2 = j << 1;
      rowSum[i] += mtr1[i][j2] * mtr1[i][j2 + 1];
    }
  }

  for (size_t i = 0; i < t; i++)
  {
    for (size_t j = 0; j < halfm; j++)
    {
      size_t j2 = j << 1;
      colSum[i] += mtr2[j2][i] * mtr2[j2 + 1][i];
    }
  }
```

```
  for (size_t i = 0; i < n; i++)
  {
    for (size_t j = 0; j < t; j++)
    {
      int temp = -rowSum[i] - colSum[j];

      for (size_t k = 0; k < halfm; k++)
      {
        size_t k2 = k << 1;
        temp += (mtr1[i][k2] + mtr2[k2 + 1][j]) *
        (mtr1[i][k2 + 1] + mtr2[k2][j]);
      }

      if (m & 1)
      {
        temp += mtr1[i][m - 1] * mtr2[m - 1][j];
      }

      res[i][j] = temp;
    }
  }

  return res;
}
```

Листинг 4.3 — Реализация оптимизированного алгоритма Винограда